



ЛЕКЦИЯ 11

Обучение без учителя и обнаружение аномалий в данных безопасности

Машинное обучение и безопасность систем

Магистратура НИУ ВШЭ (МИЭМ) · ОП «Информационная безопасность и технологии ИИ»

Автор: Силаев Ю. В. · Время изучения: ~60-70 минут



О чём эта лекция за одну минуту

2 / 53

ЧАСТЬ 1 · ВВЕДЕНИЕ

Краткая аннотация: что вы прочитаете и зачем

До сих пор курс работал с задачами, где у каждого объекта есть надёжная метка: «атака» или «норма». В реальной безопасности это роскошь. Многие подозрительные события **редки, заранее неизвестны или размечаются с задержкой**. Эта лекция – про методы, которые работают, когда полной разметки нет.

Вы познакомитесь с тремя разными инструментами: **кластеризацией** (поиск структуры), **снижением размерности** (сжатие и визуализация) и **обнаружением аномалий** (поиск необычного). Это не замена классификации, а другой класс задач – с другими ограничениями качества.

КЛЮЧЕВОЙ ТЕЗИС ЛЕКЦИИ

Аномалия не равна атаке. Методы обучения без учителя помогают находить необычное поведение, но требуют доменной интерпретации и осторожной проверки – результат – это сигнал для расследования, а не окончательный вердикт.



Учебные результаты

- различать кластеризацию, снижение размерности и обнаружение аномалий – и понимать, когда обучение без учителя уместно в задачах безопасности;
- объяснять на прикладном уровне идеи k-means, DBSCAN, PCA, UMAP/t-SNE, Isolation Forest и One-Class SVM;
- понимать, почему масштаб признаков и выбор признакового пространства напрямую влияют на результат;
- оценивать качество там, где надёжной разметки нет: precision-at-top-k и порог через нагрузку на аналитика;
- различать выброс, редкий нормальный объект и потенциально вредоносное событие;
- критически читать публичные ИБ-датасеты (NSL-KDD, CIC-IDS-2018) с их известными артефактами разметки.



Предварительные требования и явные отсылки на предыдущие лекции

Лекция опирается на материал предыдущих лекций блоков 1-2. Перед изучением убедитесь, что владеете следующим:

Объект и признаки – представление данных как объектов в признаковом пространстве $x \in \mathbb{R}^d$ (введено в L02).

Метрики качества – precision, recall, кривые ошибок и цена ошибок (L04) – здесь понадобится их частный случай.

Валидация и leakage – схемы split, утечка данных и идея group split (L05) – без переопределения.

Конвейеры и масштабирование – масштабирование признаков внутри Pipeline (L10) – критично для расстояний.

ЧТО НЕ ТРЕБУЕТСЯ

Временная структура данных не нужна. В этой лекции датасеты – это «снимки»: множества объектов без учёта порядка во времени. Sessionization, time-based split и concept drift – материал следующей лекции L12.



Семь частей · 53 слайда · ~60-70 минут самостоятельного изучения

1	Введение	1-5	~6 мин
2	Мотивация: зачем это нужно в ИБ	6-9	~5 мин
3	Основное содержание (4 блока)	10-43	~45 мин
	3.1 · Структура: кластеризация	10-19	
	3.2 · Снижение размерности	20-26	
	3.3 · Обнаружение аномалий	27-36	
	3.4 · Оценка качества без разметки	37-43	
4	Связки с практикой ИБ	44-49	~8 мин
5	Итоги	50-51	~3 мин
6	Источники для углубления	52	~1 мин
7	Глоссарий	53	~1 мин



ЧАСТЬ 2

Зачем это нужно в информационной безопасности

Реальная задача SOC · что решает обучение без учителя · цена отсутствия инструмента



Реальная задача: разметки нет или она опаздывает

7 / 53

ЧАСТЬ 2 · МОТИВАЦИЯ

Почему supervised classification буксует в типичном сценарии SOC

Представьте дежурную смену SOC. На вход поступает поток событий: сетевые соединения, входы в учётные записи, обращения к файлам. Аналитику нужно понять, есть ли среди них подозрительные. Проблема в том, что **надёжных меток «атака / норма» почти нет**: новые угрозы заранее неизвестны, разметка появляется только после расследования, а на её получение уходят дни или недели.

Supervised classification (как в L06-L08) требует обучающей выборки с метками. Когда меток для интересующего нас класса попросту нет – **не «мало», а нет вообще**, – обучить классификатор на этот класс нечем. Это не недостаток конкретной модели, а свойство задачи: мы ищем то, чего ещё не видели и не размечали.

ПОЧЕМУ МЕТОК НЕТ ИЛИ ОНИ ОПАЗДЫВАЮТ

Редкость – интересные события составляют доли процента потока. **Новизна** – неизвестная атака не присутствует ни в каком обучающем наборе. **Задержка** – метка появляется только после ручного расследования, уже постфактум.



Три разные задачи – и ни одна не является заменой классификации

Обучение без учителя не пытается предсказать заранее известную метку. Оно работает со структурой самих данных. В этой лекции мы разберём три разных инструмента, решающих три разные задачи:

ЗАДАЧА 1

Кластеризация

Группировка похожих объектов без заранее известных классов. Поиск структуры: «какие естественные скопления есть в данных?»

ЗАДАЧА 2

Снижение размерности

Сжатие многих признаков в немногие для визуализации и диагностики. «Как увидеть форму данных в 2D?»

ЗАДАЧА 3

Обнаружение аномалий

Поиск объектов, плохо согласующихся с массой остальных. «Что здесь необычно по сравнению с нормой?»

ГЛАВНОЕ ОТЛИЧИЕ

Это не замена классификации, а другой класс задач. Где есть надёжная разметка – работает supervised (L06-L08). Unsupervised применяют там, где разметки нет или она ненадёжна, и качество оценивается иначе.



Два типовых провала инженера, у которого нет рамки unsupervised

⚠ ЧАСТАЯ ОШИБКА

«Раз меток нет – будем глазами просматривать всё подряд»

Без приоритизации аналитик тонет в потоке: тысячи событий в смену, проверить можно десятки. Либо смотрят случайную долю и пропускают редкое, либо выгорают. Обратная крайность – **объявить атакой любой выброс** без проверки, завалив команду ложными тревогами.

ПОЧЕМУ ЭТО ВРЕДНО

Цена – пропущенные инциденты и потеря доверия к системе

Поток без приоритизации означает, что реальная угроза с высокой вероятностью останется непросмотренной – её просто не успеют дойти. **А поток ложных тревог обучает аналитика игнорировать сигналы:** после сотни «пустых» алертов сто первый, настоящий, тоже закроют не глядя. Инструмент, которому перестали доверять, хуже его отсутствия.

Дальше (Часть 3): как именно искать структуру и необычное в данных – кластеризация, снижение размерности, обнаружение аномалий и оценка качества без разметки.



ЧАСТЬ 3 · БЛОК 3.1

Структура данных: кластеризация

Сейчас разберём: как искать структуру в неразмеченных данных — и почему это не классификация



Первое и главное различие, которое нужно усвоить

ОПРЕДЕЛЕНИЕ

Кластеризация

Разбиение множества объектов на группы (кластеры) так, чтобы внутри группы объекты были похожи, а между группами – различны. Целевая переменная не используется: меток классов нет.

Ключевое отличие от классификации. Классификатор учится воспроизводить заранее известные метки (L06-L08): ему показывают «вот атака, вот норма». Кластеризация меток не видит – она лишь обнаруживает, что объекты естественным образом собираются в скопления. Что означают эти скопления и существуют ли они «на самом деле» – отдельный вопрос, который алгоритм не решает.

Практический смысл в ИБ: кластеризация – инструмент разведочного анализа. Она помогает увидеть группы похожих алертов, похожих сессий или похожих файлов, чтобы аналитик мог изучать их пачками, а не по одному. Это гипотеза о структуре, а не приговор о принадлежности к классу.



Признаковое пространство: почему масштаб решает всё

Расстояние между объектами – основа большинства методов этой лекции

Объект описывается вектором признаков $\mathbf{x} \in \mathbb{R}^d$. Близость объектов чаще всего измеряют евклидовым расстоянием – оно лежит в основе k-means, многих anomaly-методов и интерпретации проекций:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_i (\mathbf{x}_i - \mathbf{y}_i)^2}$$

Проблема масштаба. В сумме под корнем каждый признак вносит вклад в своих единицах. Если один признак измеряется в байтах ($0 \dots 10^9$), а другой – в долях ($0 \dots 1$), первый полностью доминирует в расстоянии, а второй почти не влияет. Алгоритм «видит» только крупномасштабный признак, каким бы информативным ни был мелкий.

ВЫВОД

Перед кластеризацией признаки масштабируют (StandardScaler / MinMaxScaler), причём внутри Pipeline – чтобы не было утечки (L05, техника – L10). Иначе результат определяет выбор единиц, а не структура данных.



Самый известный алгоритм кластеризации – и его внутренняя логика

k-means ищет **k** центров (центроидов) так, чтобы суммарное расстояние от объектов до ближайшего центра было минимальным. Число **k** задаёт человек заранее – алгоритм его не выбирает.

$$J = \sum_j \sum_{(x \in C_j)} \|x - \mu_j\|^2 \rightarrow \min$$

μ_j – центроид кластера C_j ; J – суммарный внутрикластерный разброс, который минимизируется.

ШАГ А · НАЗНАЧЕНИЕ

Каждый объект приписывается к ближайшему центроиду. Получаем k групп при текущих центрах.

ШАГ В · ПЕРЕСЧЁТ

Центроид каждой группы пересчитывается как среднее её объектов. Шаги А и В повторяются до стабилизации.



Где алгоритм даёт обманчивый результат – и о чём помнить аналитику

Форма кластеров – k-means тяготеет к компактным «шарообразным» группам сопоставимого размера. Вытянутые, изогнутые или вложенные структуры он разрезает неправильно.

Чувствительность к масштабу – без предварительного масштабирования признаков результат определяется единицами измерения, а не структурой (см. слайд 12).

Зависимость от k и инициализации – k задаётся заранее; разный старт центров даёт разные разбиения. На практике запускают многократно (k-means++, n_init) и сравнивают.

Чувствительность к выбросам – выброс смещает центроид на себя, искажая весь кластер – а в ИБ выбросы как раз и интересны.

ЧТО ИЗ ЭТОГО СЛЕДУЕТ

k-means – быстрый и понятный baseline для разведки, но не «детектор групп истины». Когда форма кластеров сложная или важны выбросы, нужен другой инструмент – к нему перейдём дальше (DBSCAN).



DBSCAN: плотностные кластеры и шум

Другой взгляд на структуру – через плотность, а не через центры

ОПРЕДЕЛЕНИЕ

DBSCAN

Алгоритм, который выделяет кластеры как области высокой плотности точек, разделённые разрежёнными зонами. Число кластеров не задаётся заранее, а точки в разрежённых зонах помечаются как шум – они не относятся ни к одному кластеру.

Два параметра задают, что считать «плотным»:

EPS (E)

Радиус окрестности точки. Насколько близко должны лежать соседи, чтобы считаться рядом.

MIN_SAMPLES

Сколько соседей нужно в радиусе ϵ , чтобы точка считалась «ядровой» и могла образовать кластер.

Три роли точек: ядровая (core) – имеет $\geq \text{min_samples}$ соседей в радиусе ϵ ; граничная (border) – попадает в окрестность ядровой, но сама соседей не набирает; шумовая (noise) – не дотягивает ни до того, ни до другого.



Как при заданных ϵ и min_samples формируются кластер и шум

УСЛОВИЕ

Параметры: $\epsilon = 1.0$, $\text{min_samples} = 3$. Семь точек на прямой (координата): $A=0.0$ $B=0.4$ $C=0.8$ $D=1.1$
 $E=1.4$ | $P=5.0$ | $Q=9.0$.

Точка	Соседей в радиусе $\epsilon=1.0$ (вкл. себя)	Роль
A (0.0)	A, B, C $\rightarrow$ 3	ядровая (core)
B (0.4)	A, B, C, D $\rightarrow$ 4	ядровая (core)
C (0.8)	A, B, C, D, E $\rightarrow$ 5	ядровая (core)
D (1.1)	B, C, D, E $\rightarrow$ 4	ядровая (core)
E (1.4)	C, D, E $\rightarrow$ 3	ядровая (core)

Точки **P (5.0)** и **Q (9.0)** имеют лишь по одному соседу (себя) $\rightarrow$ это **шум (noise)**, вне кластеров.

Итог: A-E $\rightarrow$ один кластер, P и Q $\rightarrow$ шум. Число кластеров DBSCAN определил сам – k задавать не пришлось.



Не «какой лучше», а «какой подходит этим данным»

K-MEANS

Центры и компактные группы

Нужно знать k заранее. Хорош для компактных групп сопоставимого размера; быстр и масштабируется на большие данные. Все точки попадают в какой-то кластер – понятия «шум» нет. Выбросы смещают центроиды. Хороший быстрый baseline для разведки.

DBSCAN

Плотность и произвольная форма

k знать не нужно. Находит кластеры произвольной формы и явно выделяет шум – это ценно для ИБ, где интересны именно «лишние» точки. Зато чувствителен к выбору ϵ и min_samples и плохо работает, когда плотность кластеров сильно разная.

ПРАВИЛО ВЫБОРА

Форма групп простая и нужен быстрый обзор – k-means. Форма сложная или важно отделить «шумовые» необычные точки – DBSCAN. В обоих случаях масштабируйте признаки заранее.



Частая ошибка: кластер – это не класс

Самое опасное заблуждение при работе с кластеризацией

⚠ ЧАСТАЯ ОШИБКА

«Алгоритм нашёл 4 кластера – значит, есть 4 типа трафика»

Кластеры присваивают именам сущности постфактум: «вот это сканирование, это эксфильтрация...». Но алгоритм лишь сгруппировал точки по близости в выбранном признаковом пространстве. **Он не знает ни про атаки, ни про норму** – и не утверждал, что групп именно столько.

ПОЧЕМУ ЭТО НЕВЕРНО

Кластеры зависят от признаков, масштаба и параметров

Поменяйте набор признаков, масштабирование или k – и «типы» рассыплются или сольются. Кластер может смешивать разные атаки и одновременно дробить одну норму на части. **Принятые за классы кластеры дают ложную уверенность**: решения строятся на артефакте конкретной настройки, а не на структуре угроз. Кластеры – это гипотеза для проверки экспертом, а не разметка.



Что нужно унести из блока 3.1 перед переходом к снижению размерности

- 1 **Структура, а не классы** – кластеризация ищет группы похожих объектов без меток; что они «значат» – решает эксперт, а не алгоритм.
- 2 **Масштаб критичен** – методы на расстояниях бессмысленны без масштабирования признаков – делайте его внутри Pipeline (L05/L10).
- 3 **k-means – про центроиды** – быстрый baseline для компактных групп; нужно задавать k, чувствителен к форме, масштабу и выбросам.
- 4 **DBSCAN – про плотность** – находит кластеры произвольной формы и явно выделяет шум; k не нужен, но важны ϵ и min_samples.

ДАЛЬШЕ – БЛОК 3.2

Снижение размерности (PCA, UMAP/t-SNE): как сжать многомерные данные до 2D для визуализации – и почему красивая картинка не доказывает наличие классов.



ЧАСТЬ 3 · БЛОК 3.2

Снижение размерности и визуализация

Сейчас разберём: как сжать многомерные данные до 2D – и почему картинка не доказывает наличие классов



РСА: линейное снижение размерности

Найти немного направлений, в которых данные «разбросаны» сильнее всего

Признаков может быть десятки или сотни (в CIC-IDS-2018 – 80). **РСА (метод главных компонент)** ищет новые оси – линейные комбинации исходных признаков – вдоль которых разброс данных максимален. Первая компонента ловит наибольшую дисперсию, вторая – наибольшую из оставшейся, и так далее.

$$\Sigma \mathbf{v}_k = \lambda_k \mathbf{v}_k \quad (\Sigma - \text{ковариационная матрица})$$

Главные компоненты $\mathbf{v}_k$ – собственные векторы ковариационной матрицы; λ_k – соответствующая дисперсия вдоль оси.

ЧТО ВАЖНО ПОНИМАТЬ

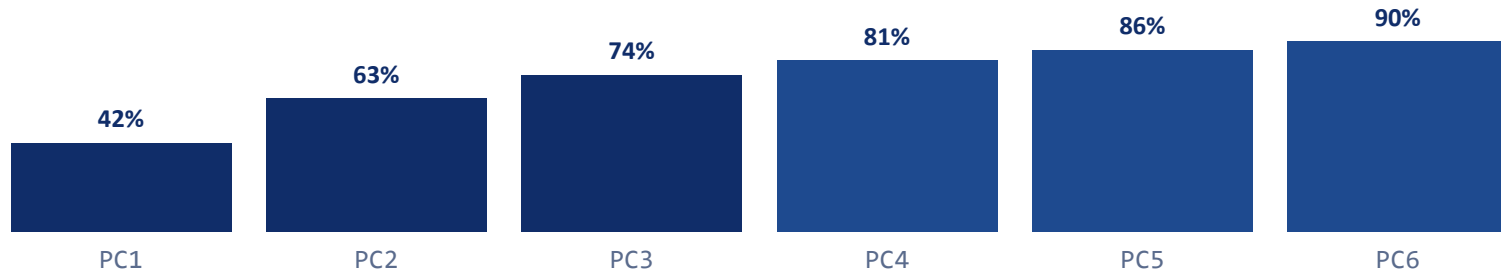
РСА – линейный метод. Он хорошо сжимает данные, лежащие около линейного подпространства, но не «разворачивает» нелинейные структуры. Признаки перед РСА масштабируют – иначе компоненты определит самый крупномасштабный признак, а не структура (как на слайде 12).



Сколько компонент нужно – и что они вообще описывают

Каждая компонента объясняет долю общей дисперсии. Накопленная объяснённая дисперсия показывает, сколько информации сохраняется при сжатии до k компонент. Это главный диагностический инструмент РСА.

Накопленная объяснённая дисперсия (пример):



КАК ЧИТАТЬ

Первые 3 компоненты здесь дают 74% дисперсии. Если кривая быстро выходит на плато – данные сжимаются хорошо; если растёт почти линейно – линейного подпространства нет, РСА малоинформативен.



UMAP и t-SNE: нелинейная визуализация

Когда линейного PCA мало – но и доверять им нужно осторожно

ОПРЕДЕЛЕНИЕ

UMAP / t-SNE

Нелинейные методы проекции в 2D-3D, которые стараются сохранить локальную структуру: точки, близкие в исходном пространстве, остаются близкими на картинке. Используются прежде всего для визуального исследования данных.

Сохраняют локальное, искажают глобальное – расстояния между далёкими группами на картинке не отражают истинных расстояний; размер и промежутки между кластерами читать нельзя.

Чувствительны к параметрам – perplexity (t-SNE), n_neighbors и min_dist (UMAP) сильно меняют вид проекции – одни и те же данные дают разные картинки.

Стохастичны – разный random_state – разный результат; форма «островов» частично случайна.



Частая ошибка: картинка как доказательство

Самое частое заблуждение при работе с проекциями

⚠ ЧАСТАЯ ОШИБКА

«На t-SNE видно 5 чётких островов – значит, есть 5 классов»

Глаз видит обособленные сгустки и достраивает вывод о «типах». Но форма, число и обособленность «островов» на нелинейной проекции – во многом **артефакт алгоритма и его параметров**, а не свойство данных.

ПОЧЕМУ ЭТО НЕВЕРНО

Поменяйте параметры – и «классы» рассыплются

Другой perplexity или random_state даст другое число и форму островов. Один настоящий класс может распасться на несколько пятен, а разные классы – слиться. **Визуализация – это гипотеза, а не доказательство**: проверяйте находку по исходным признакам и метрикам, а не по красоте проекции.



OOD-объекты: что выпадает из общей массы

Полезный сигнал, который видно даже на осторожной проекции

Out-of-distribution (OOD) объект – объект, лежащий далеко от области, где сосредоточены «нормальные» данные. На проекции такие точки часто видны как одиночки вдали от основной массы. Это полезная подсказка для дальнейшего анализа: что-то не похоже на типичную норму.

ОСТОРОЖНО

Близость к массе на 2D-проекции не гарантирует «нормальность», а удалённость – «аномальность». Проекция искажает расстояния (слайд 23). OOD-сигнал с картинки – повод проверить объект в исходном пространстве, а не вердикт.

СВЯЗЬ С КУРСОМ

OOD и сдвиг данных – большая отдельная тема. Здесь мы лишь замечаем OOD как визуальный сигнал. Систематически OOD-объекты разбираются в контексте устойчивости к сдвигу данных – это L17.



Мини-резюме: снижение размерности

Что унести из блока 3.2 перед переходом к обнаружению аномалий

- 1 PCA – линейный** – ищет оси максимальной дисперсии; объяснённая дисперсия показывает, сколько информации сохраняется при сжатии.
- 2 UMAP / t-SNE – нелинейные** – сохраняют локальную близость, но искажают глобальные расстояния и зависят от параметров и `random_state`.
- 3 Визуализация – диагностика, не доказательство** – форма и число «островов» – гипотеза для проверки, а не подтверждение классов.
- 4 OOD – сигнал, а не вердикт** – удалённые на проекции точки стоит проверить в исходном пространстве; системно – в L17.

ДАЛЬШЕ – БЛОК 3.3

Обнаружение аномалий (Isolation Forest, One-Class SVM): как искать редкое и необычное – и почему ключевой тезис лекции «аномалия ≠ атака».



ЧАСТЬ 3 · БЛОК 3.3

Обнаружение аномалий

Сейчас разберём: как искать редкое и необычное – и почему ключевой тезис лекции «аномалия ≠ атака»



И почему её нельзя сразу называть атакой

ОПРЕДЕЛЕНИЕ

Аномалия

Объект, который плохо согласуется с моделью «нормального» поведения, построенной по основной массе данных. Аномалия определяется относительно того, что принято за норму, – а не сама по себе.

КЛЮЧЕВОЙ ТЕЗИС ЛЕКЦИИ

Аномалия ≠ атака. Необычное поведение может быть атакой, но точно так же – редким легитимным событием, сбоем, изменением инфраструктуры или артефактом данных. Anomaly detection даёт сигнал для проверки, а не вердикт.

Почему так. Метод не знает, что такое «атака» – он знает только, что объект редок или непохож на остальные. Связать редкость с угрозой может лишь доменная интерпретация: тот же одиночный всплеск трафика бывает и сканированием, и ночным бэкапом.



Какой тип мы разбираем в этой лекции – и какой остаётся на L12

POINT · В ЭТОЙ ЛЕКЦИИ

Точечная

Отдельный объект необычен сам по себе, вне зависимости от соседей. Пример: соединение с аномально большим объёмом переданных байт.

CONTEXTUAL · L12

Контекстная

Объект необычен только в своём контексте (чаще временном).
Пример: вход в систему в 4 утра – норма днём, аномалия ночью.

COLLECTIVE · L12

Коллективная

Аномальна группа объектов вместе, хотя каждый по отдельности нормален. Пример: серия мелких переводов как один паттерн.

ГРАНИЦА ТЕМЫ (РАЗВИЛКА L11 ↔ L12)

В этой лекции данные – «снимок» без времени, поэтому мы работаем с точечными аномалиями. Контекстные и коллективные аномалии по-настоящему раскрываются, когда появляется временная структура: sessionization, окна, порядок событий. Это материал L12 – здесь мы их только называем.



Аномалию проще «отрезать» от остальных, чем нормальную точку

Алгоритм многократно строит случайные деревья: на каждом шаге берёт случайный признак и случайный порог, разрезая данные надвое. **Аномальные точки изолируются за меньшее число разрезов** – они лежат в разрежённых областях, поэтому быстро отделяются. Нормальные точки в плотной массе приходится резать долго.

$\text{score}(x) \uparrow$ когда средняя глубина $h(x) \downarrow$

$h(x)$ – средняя длина пути до изоляции точки x по всем деревьям; чем короче путь, тем выше anomaly score.

ЧЕМ ХОРОШ НА ПРАКТИКЕ

Быстрый, масштабируется на большие данные, не требует задавать «форму» нормы. Параметр contamination задаёт ожидаемую долю аномалий – но это предположение аналитика, а не факт из данных.



Почему выброс отделяется за меньшее число разрезов

ДАННЫЕ

Один признак, шесть точек: 10, 11, 12, 13, 14 – плотная норма; и 100 – явный выброс. Делаем случайные разрезы (порог между min и max) и считаем, за сколько шагов точка осталась одна.

Точка	Типичный путь до изоляции	Глубина h
100 (выброс)	первый же разрез ~50 отрезает 100 от всех	≈ 1 – мелко
10, 14 (края нормы)	несколько разрезов внутри плотной зоны	≈ 3-4
12 (центр нормы)	режется дольше всех – со всех сторон соседи	≈ 4-5 – глубоко

Итог: у выброса 100 путь короче → выше anomaly score. Isolation Forest не описывает «норму» явно – он эксплуатирует то, что редкое отрезается легче.



Другой подход: очертить область нормального поведения

One-Class SVM обучается на данных, которые считаются нормальными, и строит **границу вокруг области нормы**. Всё, что попадает внутрь, — нормально; всё снаружи — кандидат в аномалии. В отличие от Isolation Forest, метод явно моделирует «форму» нормального множества.

$f(x) \geq 0 \rightarrow$ норма

$f(x) < 0 \rightarrow$ аномалия

Граница задаётся опорными объектами; ядро (RBF) позволяет ей быть нелинейной, ν задаёт долю «выходов» за границу.

О ЧЁМ ПОМНИТЬ

Чувствителен к параметрам (ν , γ) и к масштабу признаков; тяжелее на больших данных, чем Isolation Forest. Если «нормальные» обучающие данные на деле содержат атаки, граница вберёт их в норму.



Isolation Forest против One-Class SVM – два способа отделить редкое от обычного

ISOLATION FOREST

Изолирует редкое

Быстрый, хорошо масштабируется, мало параметров.
Не моделирует норму явно – ищет точки, которые легко отрезать. Устойчив в высокой размерности.
Параметр contamination – доля аномалий – это предположение, а не факт.

ONE-CLASS SVM

Очерчивает норму

Строит границу области нормы (RBF – нелинейную).
Гибче по форме границы, но чувствителен к v и γ , к масштабу и хуже масштабируется на большие выборки. Требует «чистых» нормальных данных для обучения.

ПРАВИЛО ВЫБОРА

Много данных, высокая размерность, нужен быстрый baseline – Isolation Forest. Есть относительно чистая «норма» и важна гибкая граница – One-Class SVM. В обоих случаях масштабируйте признаки.



Три разные сущности, которые легко смешать

Выброс, редкое нормальное и вредоносное – это не одно и то же

СТАТИСТИКА

Выброс

Объект, далёкий от массы в признаковом пространстве. Чисто статистическое свойство – ничего не говорит о причине.

ЛЕГИТИМНОЕ

Редкое нормальное

Настоящее, но нечастое поведение: ночной бэкап, редкий отчёт, разовая массовая рассылка. Выброс – но не угроза.

УГРОЗА

Вредоносное

Событие, связанное с атакой. Может быть выбросом – а может прятаться среди нормы и выбросом не быть вовсе.

ВЫВОД

Anomaly detection находит выбросы. Является ли выброс редким нормальным или вредоносным – решает доменная проверка. И наоборот: не всякое вредоносное является выбросом, поэтому anomaly detection не ловит «тихие» атаки внутри нормы.



Ошибка: аномалии вместо классификации

Когда unsupervised применяют там, где уже есть разметка

⚠ ЧАСТАЯ ОШИБКА

«Возьмём anomaly detection – это же про безопасность»

Есть размеченные примеры атак и нормы – но инженер по привычке запускает Isolation Forest, потому что «ищем подозрительное». **Размеченные данные при этом простаивают**, а метод ищет любую необычность, а не известные атаки.

ПОЧЕМУ ЭТО НЕВЕРНО

При достаточной разметке нужен supervised

Классификатор (L06-L08), обученный на метках, прицельно отделяет известные атаки от нормы и оценивается честными метриками (L04). Anomaly detection не использует метки и не оптимизирует под них – **его качество на той же задаче будет ниже**. Unsupervised – для случаев, когда меток нет; не нужно подменять им классификацию.



Мини-резюме: обнаружение аномалий

Что унести из блока 3.3 перед переходом к оценке качества

- 1 **Аномалия ≠ атака** – методы находят необычное относительно нормы; связь с угрозой устанавливает доменная проверка, а не алгоритм.
- 2 **Isolation Forest – изолирует редкое** – редкие точки отрезаются за меньшее число разрезов; быстрый, масштабируемый, мало параметров.
- 3 **One-Class SVM – очерчивает норму** – строит границу нормального множества; гибче, но чувствителен к параметрам, масштабу и чистоте обучающих данных.
- 4 **Три сущности различать** – выброс ≠ редкое нормальное ≠ вредоносное; не всякий выброс опасен и не всякая атака – выброс.

ДАЛЬШЕ – БЛОК 3.4

Оценка качества без разметки: как понять, что детектор работает, если меток нет – precision-at-top-k и порог через нагрузку на аналитика.



ЧАСТЬ 3 · БЛОК 3.4

Оценка качества без разметки

Сейчас разберём: как понять, что детектор работает, если надёжных меток нет



Почему привычные метрики здесь не работают

Все метрики из L04 – accuracy, precision, recall, ROC-AUC – требуют **истинных меток**: с чем сравнивать предсказание. В задачах без учителя меток нет или они ненадёжны, поэтому посчитать эти метрики честно нельзя. Любое «accuracy = 0.98» здесь – иллюзия: оно опирается на разметку, которой у нас на самом деле нет.

ЛОВУШКА ЛОЖНОЙ УВЕРЕННОСТИ

Считать accuracy относительно «псевдо-меток» (например, кластерных) – значит проверять модель на её же догадках. Высокая цифра при этом ничего не доказывает.

Что делать. Раз полной разметки нет, оценка строится на ограниченной ручной проверке: эксперт смотрит небольшую часть результатов и подтверждает, насколько они осмысленны. Отсюда – две практические техники на следующих слайдах.



Главная практическая метрика, когда полной разметки нет

Детектор выдаёт каждому объекту anomaly score. Берём **k объектов с наивысшим score** – те, что эксперт реально успеет проверить руками – и смотрим, какая доля из них действительно оказалась подозрительной.

$$P@k = (\text{истинно подозрительных среди top-k}) / k$$

ПРИМЕР

Аналитик проверяет top-20 по score. Из них 14 – реальные инциденты. $P@20 = 14 / 20 = 0.70$. Метрика требует разметить только эти 20 объектов, а не весь датасет.

Связь с L04: общие метрики качества вводятся там; precision-at-top-k – их частный случай для приоритизации при ручной проверке.



Сколько алертов поднимать – решает не «магия», а ёмкость проверки

Детектор даёт score, но решение «аномалия / норма» появляется только после **порога**. Откуда его взять без разметки? От практического ограничения: сколько объектов команда физически способна проверить за смену.

ЛОГИКА ВЫБОРА ПОРОГА

Аналитик проверяет ~50 объектов в день → порог ставим так, чтобы выше него оказалось около 50 самых подозрительных. Порог задаёт допустимую нагрузку, а не абстрактную «вероятность атаки».

СВЯЗЬ С КУРСОМ (ЦЕНА ОШИБОК И ПОРОГ)

Это та же линия «цена ошибок и порог», что в L04 и L07. Слишком низкий порог – поток ложных тревог и выгорание; слишком высокий – пропущенные инциденты. Баланс задаёт ёмкость проверки и цена каждого типа ошибки.



Даже без меток случайный split способен обмануть

Если оценивать детектор на тех же сущностях, что участвовали в его настройке, **оценка окажется завышенной**. Объекты одного пользователя, хоста или сессии похожи между собой; попав и в «настройку», и в «проверку», они создают видимость качества, которой не будет на новых сущностях.

GROUP SPLIT – КАК В L05

Разделяйте данные по группам (пользователь / хост / сессия), а не по отдельным объектам. Тогда проверка идёт на сущностях, которых детектор «не видел» – это честнее отражает поведение на новых данных.

ГРАНИЦА ТЕМЫ

Здесь – только группа. Временной split (обучение на прошлом, проверка на будущем) – это уже временная структура, материал L12.



Частая ошибка: accuracy на unsupervised

Как незаметно обмануть себя при оценке детектора

⚠ ЧАСТАЯ ОШИБКА

«Посчитаю accuracy – 0.99, отличная модель»

Аномалий доля процента, поэтому модель, объявляющая всё нормой, даёт accuracy ≈ 0.99 – и не находит ни одной атаки. **Высокая accuracy здесь означает бесполезность, а не качество.**

ПОЧЕМУ ЭТО НЕВЕРНО

Accuracy здесь неприменима или вводит в заблуждение

Считать accuracy не на чем (меток нет) – или она доминируется огромным классом нормы и ничего не говорит о редком. **Оценивайте precision-at-top-k и осмысленность top-объектов при ручной проверке, а порог задавайте через нагрузку аналитика.**



Мини-резюме: оценка без разметки

ЧАСТЬ 3.4 · ОЦЕНКА БЕЗ РАЗМЕТКИ

Что унести из блока 3.4 – и из всей части 3

- 1 **Нет меток – нет accuracy** – привычные метрики требуют истинных меток; без них честно посчитать их нельзя, а псевдо-метки дают ложную уверенность.
- 2 **Precision-at-top-k** – проверяем долю истинно подозрительных среди k объектов с наивысшим score – ровно тех, что эксперт успеет посмотреть.
- 3 **Порог – через нагрузку** – порог задаёт допустимое число объектов на ручную проверку, а не абстрактную «вероятность атаки» (линия L04 / L07).
- 4 **Group split, как в L05** – делите по группам (пользователь / хост / сессия), иначе оценка завышена; временной split – это L12.

ДАЛЬШЕ – ЧАСТЬ 4

Связки с практикой ИБ: сквозной датасет CIC-IDS-2018, прогон unsupervised на flow-признаках и критический разбор известных артефактов публичных ИБ-датасетов.



Ч А С Т Ь 4

Связки с практикой ИБ

Сквозной датасет CIC-IDS-2018 · unsupervised на flow-признаках · известные артефакты публичных ИБ-датасетов



Карточка-напоминание о датасете, который проходит через весь курс

ИБ-КЕЙС · CIC-IDS-2018: ЧТО ЭТО

Публичный датасет сетевого трафика: ≈ 16 млн flow-записей за 10 дней, 80 признаков на каждый поток, 14 типов атак и класс Benign. Признаки – агрегаты потока: длительность, число и размер пакетов, флаги, скорости. В этой лекции датасет используется как «снимок»: множество объектов в признаковом пространстве, без учёта времени.

Почему он сквозной. Одна и та же задача – обнаружение подозрительного трафика – решается на нём всё более продвинутыми инструментами по ходу курса (L02, L04, L05, L07-L10, здесь, L12, L17, L18). Студент видит узнаваемый датасет и преемственность методов.

Развилка с L12: временной порядок flow-записей здесь сознательно игнорируется – он появляется в L12.



Как unsupervised-прогон выглядит в работе аналитика

ИБ-КЕЙС · ПОСТАНОВКА

Берём «снимок» flow-записей CIC-IDS-2018, масштабируем признаки, запускаем Isolation Forest. Меток при анализе не используем – имитируем ситуацию, где разметки нет. Сортируем потоки по anomaly score и отдаём аналитику top-k на проверку.

Что в top-k по score	Интерпретация аналитика
Потоки с очень большим объёмом байт	часть – эксфильтрация, часть – легальный бэкап
Множество коротких соединений к разным портам	похоже на сканирование – проверить источник
Редкий, но штатный сервисный трафик	ложное срабатывание – это норма

Аномалия ≠ атака на практике: в одном и том же top-k оказались и реальная атака, и легитимная редкость. Решение о каждом потоке принимает эксперт, а не score.



Линия курса: ИБ-датасеты и их ограничения (L02 → L05 → L11 → L17)

Высокое качество на публичном датасете часто означает не силу метода, а **артефакт самого датасета**. Известные проблемы NSL-KDD / KDD-99 и CIC-IDS-2018:

Артефакты разметки и сбора – атаки и норма различаются по «техническим» признакам (длительность, TTL), а не по сути атаки – задача становится нереалистично лёгкой.

Дисбаланс и устаревание – распределение классов и типы атак не соответствуют сегодняшней эксплуатации; KDD-99 описывает трафик 1990-х.

Нерепрезентативность – синтетическая или лабораторная генерация трафика не повторяет реальную сеть организации.

КАК ЧИТАТЬ ТАКИЕ РЕЗУЛЬТАТЫ

Цифра на публичном датасете – это верхняя, оптимистичная оценка. Реальное качество в эксплуатации почти всегда ниже; проверяйте метод на своих данных.



Ошибка: высокий скор = готовый продукт

Перенос результата с публичного датасета в эксплуатацию

⚠ ЧАСТАЯ ОШИБКА

«На NSL-KDD получили 0.99 – метод готов к продакшену»

Отличная цифра на публичном датасете воспринимается как доказательство пригодности. **Но она измеряет качество на артефактах датасета**, а не на реальном трафике, который метод увидит в бою.

ПОЧЕМУ ЭТО НЕВЕРНО

Реальная сеть устроена иначе

Изменение инфраструктуры (новый сервис, обновление, миграция) выглядит для детектора как массовая аномалия и заваливает аналитика ложными тревогами. А артефакт датасета, на котором метод «блестел», в проде отсутствует. **Публичный результат – гипотеза, требующая проверки на собственных данных** (продолжение – под drift в L17).



Особенности, которые меняют поведение unsupervised-методов в безопасности

«Норма» определяется обучающими данными – если в обучении уже были атаки, метод сочтёт их частью нормы и перестанет их замечать.

Редкое легитимное даёт ложные тревоги – ночной бэкап, разовая рассылка, новый сервис – выбросы, но не угрозы; их доля определяет нагрузку.

Изменение инфраструктуры = массовая аномалия – обновление или миграция сдвигают распределение, и детектор «звонит» на всё сразу.

Результат – приоритизация, не автоблок – unsupervised-сигнал направляет расследование; автоматически блокировать по нему опасно.

ГЛАВНОЕ

Unsupervised в ИБ – инструмент приоритизации расследования, а не автоматический арбитр. Качество зависит от того, что принято за «норму», и проверяется доменной экспертизой.



Семь ключевых утверждений

- 1 Аномалия $\neq$ атака: unsupervised находит необычное, связь с угрозой устанавливает эксперт.
- 2 Кластер $\neq$ класс: кластеризация ищет структуру, а не истинные классы.
- 3 Масштаб решает: методы на расстояниях требуют масштабирования признаков (внутри Pipeline).
- 4 Визуализация – диагностика, не доказательство: форма «островов» зависит от параметров.
- 5 Без меток: precision-at-top-k и порог через нагрузку аналитика, а не accuracy.
- 6 Публичные ИБ-дасеты обманчивы: высокий скор может быть артефактом, а не качеством.
- 7 Результат – сигнал для расследования и приоритизации, а не автоматический вердикт.



Границы метода и навигация по курсу для самостоятельного изучения

КОГДА УМЕСТНО

Меток нет или ненадёжны; нужен разведочный анализ, поиск необычного, группировка похожего, приоритизация для аналитика.

КОГДА НЕ СТОИТ

Есть надёжная разметка (нужен supervised, L06-L08); требуется автоматическая блокировка по одному сигналу; нужен вердикт без проверки.

Что читать дальше:

- L12** – временная структура: contextual / collective аномалии, sessionization, time-based split, drift.
- L17** – OOD и устойчивость к сдвигу данных – продолжение темы аномальных и нетипичных объектов.
- L04 / L07** – общие метрики и выбор порога под цену ошибок – фундамент для precision-at-top-k.
- L05 / L19** – group split и leakage; аномальные training-точки как граничный случай.



Обязательное, дополнительное и практическое

Обязательное:

- Hastie, Tibshirani, Friedman. The Elements of Statistical Learning – гл. 14 (Unsupervised Learning): кластеризация, k-means.
- Bishop. Pattern Recognition and Machine Learning – гл. 12 (PCA); гл. 9 (mixture models) – как контекст.

Дополнительно:

- Liu, Ting, Zhou. Isolation Forest (ICDM 2008) – оригинальная работа по методу.
- McInnes, Healy, Melville. UMAP (2018) – нелинейная проекция и её свойства.

Практика и данные:

- scikit-learn User Guide: Clustering, IsolationForest, OneClassSVM, decomposition.PCA.
- Датасеты: CIC-IDS-2018, NSL-KDD – с обязательной поправкой на известные артефакты (см. слайд 47).



Кластеризация	разбиение объектов на группы похожих без меток классов
k-means	кластеризация через центроиды; k задаётся заранее
DBSCAN	плотностная кластеризация; выделяет шум, k не нужен
eps, min_samples	радиус окрестности и порог соседства в DBSCAN
PCA	линейное снижение размерности по осям максимальной дисперсии
UMAP / t-SNE	нелинейная проекция для визуализации; искажает глобальное

Аномалия	объект, плохо согласующийся с моделью нормы
Isolation Forest	anomaly detection через лёгкость изоляции редких точек
One-Class SVM	граница области нормального поведения
Anomaly score	числовая мера необычности объекта
Precision-at-top-k	доля истинно подозрительных среди k объектов с высшим score
OOD	out-of-distribution: объект вне области нормальных данных